LAPORAN PRATIKUM PEKAN 3
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Stack*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan Stack.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 18 April 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN 4

1.1 Latar Belakang 4

1.2 Tujuan 4

1.3 Manfaat 4

BAB II PEMBAHASAN 5

2.1 Stack 5

2.1.1 Kode Program 5

2.1.2 Uraian Kode Program 5

2.1.3 Output 6

2.2 Stack – Nilai Maksimum 6

2.2.1 Kode Program 6

2.2.2 Uraian Kode Program 6

2.2.3 Output 7

2.3 Stack - Siswa 8

2.3.1 Kode Program 8

2.3.2 Uraian Kode Program 9

2.3.3 Output 9

2.4 Stack – StackArray 10

2.4.1 Kode Program 10

2.4.2 Uraian Kode Program 11

2.5 Stack – StackArrayDriver 12

2.5.1 Kode Program 12

2.5.2 Uraian Kode Program 12

2.5.3 Output 12

2.6 Stack – StackPostFix 13

2.6.1 Kode Program 13

2.6.2 Uraian Kode Program 13

2.6.3 Output 14

BAB III KESIMPULAN 15

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data *Stack* (tumpukan) beroperasi berdasarkan prinsip LIFO (*Last In, First Out*), di mana elemen data yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Salah satu cara fundamental untuk mengimplementasikan *Stack* adalah dengan menggunakan struktur data memori statis seperti *Array*, yang menuntut pemrogram untuk mengelola penunjuk indeks (seperti variabel top) secara manual guna melacak keberadaan elemen teratas. Selain operasi dasarnya, *Stack* memiliki peran krusial dalam arsitektur komputasi, salah satunya adalah untuk mengevaluasi ekspresi matematika berformat *Postfix* (*Reverse Polish Notation*). Praktikum ini bertujuan untuk memberikan pengalaman langsung dalam merancang mesin *Stack* dari awal (*from scratch*) menggunakan *Array*, serta menerapkan struktur data tersebut untuk menyelesaikan masalah komputasi nyata yaitu membaca dan menghitung notasi *Postfix*.

1.2 Tujuan

1. Memahami konsep dasar, karakteristik mekanis LIFO (*Last In, First Out*), dan cara kerja struktur data *Stack* di dalam memori komputer.
2. Mampu mengimplementasikan logika operasi-operasi dasar *Stack* (seperti push, pop, peek, serta pengecekan kondisi *overflow/underflow*) secara manual menggunakan tipe data *Array* tunggal.
3. Mampu menerapkan struktur data *Stack* yang telah dibuat untuk memecahkan permasalahan algoritmik, secara khusus dalam membaca dan mengevaluasi hasil akhir dari ekspresi matematika berformat *Postfix*

1.3 Manfaat

1. Meningkatkan pemahaman logika algoritmik.
2. Mengembangkan kemampuan analisis *error handling* dalam mengelola batas memori statis, khususnya dalam mencegah dan menangani kondisi *Stack Overflow* (saat kapasitas penuh) maupun *Stack Underflow* (saat tumpukan kosong).
3. Memberikan wawasan aplikatif tentang bagaimana mesin komputer modern dan *compiler* memproses operasi matematika yang kompleks secara efisien menggunakan tumpukan memori dan notasi *Postfix*.

BAB II
PEMBAHASAN

2.1 Stack

2.1.1 Kode Program

```java
package Pekan3_2511533002;

import java.util.Stack;

public class LatihanStack_2511533002 {
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(42);
        s.push(-3);
        s.push(17);
        System.out.println("Nilai stack = "+ s);
        System.out.println("Nilai pop = "+s.pop());
        System.out.println("Nilai stack setelah pop = "+s);
        System.out.println("Nilai peek"+s.peek());
        System.out.println("Nilai stack setelah peek = "+s);
    }
}
```

2.1.2 Uraian Kode Program

- Stack<Integer> s = new Stack<Integer>(); : Membuat sebuah objek tumpukan baru yang diberi nama s. Tanda <Integer> berarti tumpukan ini khusus hanya bisa diisi oleh angka bulat.
- s.push(42); : Perintah push berfungsi untuk memasukkan data ke dalam tumpukan. Angka 42 dimasukkan pertama kali, sehingga ia berada di tumpukan paling bawah.
- s.push(-3); : Angka 3 dimasukkan ke dalam tumpukan. Posisi -3 sekarang berada di atas angka 42.
- s.push(17); : Angka 17 dimasukkan ke dalam tumpukan. Sekarang posisi 17 berada di paling atas.
- System.out.println("Nilai stack = "+ s); : Mencetak seluruh isi tumpukan s ke layar.
- System.out.println("Nilai pop = "+s.pop()); : Perintah pop() berfungsi sebagai mengambil atau menghapus elemen paling atas dari tumpukan, lalu mengembalikan nilai tersebut. Karena 17 adalah yang teratas, maka 17 akan dihapus dari tumpukan dan dicetak ke layar.
- System.out.println("Nilai stack setelah pop = "+s); : Mencetak kembali isi tumpukan setelah di-pop.
- System.out.println("Nilai peek"+s.peek()); : Perintah peek() berfungsi untuk melihat elemen paling atas dari tumpukan tanpa menghapusnya
- System.out.println("Nilai stack setelah peek = "+s); : Mencetak isi tumpukan lagi untuk membuktikan bahwa perintah peek tidak menghapus data.

2.1.3 Output

```
Nilai stack = [42, -3, 17]
Nilai pop = 17
Nilai stack setelah pop = [42, -3]
Nilai peek-3
Nilai stack setelah peek = [42, -3]
```

2.2 Stack – Nilai Maksimum

2.2.1 Kode Program

```java
package Pekan3_2511533002;
import java.util.Stack;

public class NilaiMaksimum_2511533002 {
    public static int max(Stack<Integer>s) {
        Stack<Integer> backup = new Stack<Integer>();
        int Maxvalue = s.pop();
        backup.push(Maxvalue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            Maxvalue = Math.max(Maxvalue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return Maxvalue;
    }
    public static void main(String[] Args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("Isi stack "+s);
        System.out.println("Stack teratas "+s.peek());
        System.out.println("Nilai maksimum "+max(s));
    }
}
```

2.2.2 Uraian Kode Program

- public static int max(Stack<Integer>s) { : Metode ini menerima tumpukan angka (Stack<Integer>s) akan mengembalikan satu angka bulat sebagai hasil akhirnya.
- Stack<Integer> backup = new Stack<Integer>(); : membuat tumpukan baru bernama backup. Tumpukan ini bertugas sebagai tempat penitipan sementara agar data dari s tidak hilang.

- int Maxvalue = s.pop();) : mengambil pop elemen paling atas dari s, lalu untuk sementara angka ini sebagai nilai terbesar (Maxvalue).
- backup.push(Maxvalue); : Angka yang baru saja kita ambil tadi, langsung kita simpan ke tumpukan backup agar tidak hilang.
- while (!s.isEmpty()) { : Selama tumpukan utama s belum kosong (!isEmpty), lakukan perintah di bawah.
- int next = s.pop(); : Ambil angka berikutnya dari tumpukan s dan simpan di variabel next.
- backup.push(next); : Langsung titipkan angka next tersebut ke tumpukan backup.
- Maxvalue = Math.max(Maxvalue, next); : membandingkan Maxvalue dengan angka yang baru diambil.
- while (!backup.isEmpty()) { : Mulai perulangan kedua.
- s.push(backup.pop()); : Ambil angka teratas dari backup, lalu masukkan kembali ke tumpukan utama s.
- Stack<Integer> s = new Stack<Integer>(); : Membuat tumpukan utama s yang baru.
- s.push(70); : Memasukkan 70.
- s.push(12); : Memasukkan 12 di atasnya.
- s.push(20); : Memasukkan 20 di posisi paling atas.
- System.out.println("Isi stack "+s); : Mencetak isi tumpukan.
- System.out.println("Stack teratas "+s.peek()); : Mengecek elemen teratas tanpa menghapusnya.
- System.out.println("Nilai maksimum "+max(s)); : Memanggil metode max yang kita buat, lalu mencetak hasil kerja metode tersebut.

2.2.3 Output

```
Isi stack [70, 12, 20]
Stack teratas 20
Nilai maksimum 70
```

2.3 Stack - Siswa

2.3.1 Kode Program

```java
package Pekan3_2511533002;
import java.util.ArrayList;
class Siswa_2511533002 {
    String nama;
    int nim;

    public Siswa_2511533002(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "NIM : " + nim + ", Nama : " + nama;
    }
}
public class SiswaStack_2511533002 {
    private ArrayList<Siswa_2511533002> stack;

    public SiswaStack_2511533002() {
        stack = new ArrayList<>();
    }
    public void push(Siswa_2511533002 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511533002 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size()-1);
        }
        return null;
    }

    public Siswa_2511533002 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size()-1);
        }
        return null;
```

```java
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_2511533002() {
        for (int i = stack.size()-1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }
    public static void main(String[] Args) {
        SiswaStack_2511533002 studentStack = new SiswaStack_2511533002();

        Siswa_2511533002 mhs1 = new Siswa_2511533002("Ali", 1);
        Siswa_2511533002 mhs2 = new Siswa_2511533002("Boby", 2);
        Siswa_2511533002 mhs3 = new Siswa_2511533002("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack : ");
        studentStack.tampilkanSiswa_2511533002();

        System.out.println("Siswa teratas " + studentStack.peek());
        System.out.println("Mengeluarkan siswa teratas dari stack : " + studentStack.pop());
        System.out.println("Daftar siswa setelah di pop : ");
        studentStack.tampilkanSiswa_2511533002();
    }
}
```

2.3.2 Uraian Kode Program

- public void push(Siswa_2511533002 mhs) { stack.add(mhs); } : Perintah ini otomatis akan menaruh siswa tersebut di urutan paling atas.
- if (!isEmpty()) : Sebelum mengambil, pastikan dulu tumpukannya tidak kosong.
- return stack.remove(stack.size()-1); : Ini adalah cara menghapus elemen paling atas di ArrayList.
- return null; : Jika tumpukannya kosong, jangan lakukan apa-apa, kembalikan nilai kosong.
- for (int i = stack.size()-1; i >= 0; i--) { : Karena ini tumpukan dari atas ke bawah, perulangannya dimulai dari indeks paling akhir lalu terus berkurang sampai menyentuh indeks 0.
- Siswa_2511533002 mhs1 = new Siswa_2511533002("Ali", 1); : Membuat objek siswa bernama Ali.
- studentStack.push(mhs1); : Memasukkan siswa secara berurutan.
- studentStack.tampilkanSiswa_2511533002();: Mencetak tumpukan.
- System.out.println(... studentStack.peek()); : Melihat tumpukan teratas.
- System.out.println(... studentStack.pop()); : Mengeluarkan/membuang tumpukan teratas.

2.3.3 Output

```
Siswa di dalam stack :
NIM : 3, Nama : Charles
NIM : 2, Nama : Boby
NIM : 1, Nama : Ali
Siswa teratas NIM : 3, Nama : Charles
Mengeluarkan siswa teratas dari stack : NIM : 3, Nama : Charles
Daftar siswa setelah di pop :
NIM : 2, Nama : Boby
NIM : 1, Nama : Ali
```

2.4 Stack – StackArray

2.4.1 Kode Program

```java
package Pekan3_2511533002;
public class StackArray_2511533002 {
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];
    boolean isEmpty()
    {
        return (top < 0);
    }
    StackArray_2511533002()
    {
        top = -1;
    }
    boolean push(int x)
    {
        if (top >= (MAX - 1)) {
            System.out.println("Stack overflow");
            return false;
        }
    else {
        a[++top] = x;
        System.out.println(x + " dimasukkan dalam stack");
        return true;
    }
}
int pop()
{
    if (top < 0) {
        System.out.println("Stack underflow");
        return 0;
    }
    else {
        int x = a[top--];
        return x;
    }
}
int peek()
```

```java
{
    if(top < 0) {
        System.out.println("Stack underflow");
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}
void print() {
    for(int i = top;i>-1;i--) {
        System.out.println(" "+a[i]);
    }
}
}
```

2.4.2 Uraian Kode Program

- static final int MAX = 1000; : Membuat konstanta batas maksimal. Artinya, Array/Stack ini mempunyai batas maksimal hanya bisa diisi maksimal 1000 angka. Tidak bisa lebih.
- int top; : kursor yang akan menunjuk di indeks ke berapa elemen teratas tumpukan saat ini berada.
- int a[] = new int[MAX]; : Membuat sebuah Array bernama a yang memiliki 1000 slot kosong (dari indeks 0 sampai 999).
- boolean isEmpty() { return (top < 0); } : Fungsi untuk mengecek apakah tumpukan kosong.
- StackArray_2511533002() { top = -1; } :]Saat Stack pertama kali diciptakan, posisi kursor top diatur di -1. Dengan memberi nilai -1, kita menandakan bahwa array belum diisi.
- boolean push(int x) { : Fungsi untuk memasukkan angka x ke tumpukan.
- if (top >= (MAX - 1)) { : Jika kursor top sudah mencapai batas maksimal indeks (yaitu 999), berarti array penuh.
- System.out.println("Stack overflow"); return false;: Cetak peringatan dan batalkan proses (mengembalikan nilai false).
- a[++top] = x; : naikkan nilai top sebanyak 1 angka (misal dari -1 menjadi 0), lalu masukkan data x ke dalam array a pada indeks yang baru tersebut.
- System.out.println(x + " dimasukkan dalam stack"); return true; } : Cetak konfirmasi dan laporkan proses berhasil.
- int pop() { : Fungsi untuk mengambil angka teratas.
- if (top < 0) { : Mengecek kondisi Underflow. Jika top masih -1, berarti tumpukannya tidak ada isinya.
- System.out.println("Stack underflow"); return 0; } : Cetak peringatan dan keluarkan angka 0 sebagai error code.
- int x = a[top--]; : Ambil nilai array pada indeks top saat ini, simpan di wadah x, lalu kurangi nilai top sebanyak 1.
- void print() { : Fungsi untuk mencetak seluruh isi.
- for(int i = top; i > -1; i--) { : Perulangan berjalan secara mundur. Dimulai dari angka i sebesar top saat ini, terus turun (i--) sampai indeks 0.
- System.out.println(" "+a[i]); } : Cetak isi array a pada indeks ke-i ke layar monitor.

2.5 Stack – StackArrayDriver

2.5.1 Kode Program

```java
package Pekan3_2511533002;

public class StackArrayDriver_2511533002 {
    public static void main(String[] Args) {
        StackArray_2511533002 s = new StackArray_2511533002();
        s.push(10);
        s.push(20);
        s.push(30);
        System.out.println(s.pop() + " dikeluarkan dari stack");
        System.out.println("elemen teratas adalah : "+ s.peek());
        System.out.println("elemen pada stack : ");
        s.print();
    }
}
```

2.5.2 Uraian Kode Program

- s.push(10); : Memasukkan angka 10. Kursor top naik menjadi 0. Array pada indeks [0] diisi angka 10.
- s.push(20); : Memasukkan angka 20. Kursor top naik menjadi 1. Array pada indeks [1] diisi angka 20.
- s.push(30); : Memasukkan angka 30. Kursor top naik menjadi 2. Array pada indeks [2] diisi angka 30. Layar mencetak: 4. Memanipulasi Data (Pop, Peek, Print)
- System.out.println(s.pop() + " dikeluarkan dari stack"); : Perintah s.pop() mengambil elemen paling atas (yaitu 30) dan menurunkan kursor top kembali menjadi 1. Angka 30 yang terambil itu langsung digabung dengan teks dan dicetak menjadi: "30 dikeluarkan dari stack".
- System.out.println("elemen teratas adalah : "+ s.peek()); : Perintah s.peek() hanya mengintip data di kursor top saat ini (yaitu indeks 1). Datanya adalah 20. Layar akan mencetak: "elemen teratas adalah : 20". Kursor top tidak berubah, tetap di angka 1.
- System.out.println("elemen pada stack : "); : Hanya mencetak teks biasa sebagai judul/header.
- s.print(); : Memanggil fungsi cetak. Ia akan melakukan perulangan mundur mulai dari top saat ini (indeks 1) sampai ke indeks 0.

2.5.3 Output

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
elemen teratas adalah : 20
elemen pada stack :
 20
 10
```

2.6 Stack – StackPostFix

2.6.1 Kode Program

```java
package Pekan3_2511533002;
import java.util.Scanner;
import java.util.Stack;
public class StackPostfix_2511533002 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input_2511533002 = new Scanner(expression);
        while (input_2511533002.hasNext()) {
            if(input_2511533002.hasNextInt()) {// an operator (integer)
                s.push(input_2511533002.nextInt());
            }else {// an operator
                String operator_2511533002 = input_2511533002.next();
                int operatorand2_2511533002 = s.pop();
                int operatorand1_2511533002 = s.pop();
                if (operator_2511533002.equals("+")) {
                    s.push(operatorand1_2511533002 + operatorand2_2511533002);
                }else if(operator_2511533002.equals("-")) {
                    s.push(operatorand1_2511533002 + operatorand2_2511533002);
                }else if(operator_2511533002.equals("*")) {
                    s.push(operatorand1_2511533002 * operatorand2_2511533002);
                }else {
                    s.push(operatorand1_2511533002 / operatorand2_2511533002);
                }
            }
        }
        input_2511533002.close();
        return s.pop();
    }
    public static void main(String[] Args) {
        System.out.println("hasil postfix = "+postfixEvaluate("5 2 4 * + 7 -"));
    }
}
```

2.6.2 Uraian Kode Program

- Stack<Integer> s = new Stack<Integer>(); : Membuat tumpukan kosong untuk menyimpan angka.
- while (input_2511533002.hasNext()) {: Mulai perulangan: "Selama masih ada karakter/kata yang bisa dibaca, lakukan:"
- if(input_2511533002.hasNextInt()) { : Mengecek apakah kata yang sedang dibaca saat ini adalah Angka (Integer/Operand).
- s.push(input_2511533002.nextInt()); : Jika iya, ambil angka tersebut dan masukkan (push) ke dalam tumpukan s.
- String operator_2511533002 = input_2511533002.next(); : Ambil tanda baca tersebut dan simpan dalam bentuk teks.

- int operatorand2_2511533002 = s.pop(); : Ambil angka teratas dari tumpukan, jadikan sebagai angka yang posisinya di belakang.
- int operatorand1_2511533002 = s.pop(); : Ambil angka kedua dari tumpukan, jadikan sebagai angka yang posisinya di depan.
- if (operator_2511533002.equals("+")) {

  s.push(operatorand1_2511533002 + operatorand2_2511533002); } : Jika tandanya +, tambahkan kedua angka, lalu masukkan lagi hasilnya ke tumpukan.
- else if(operator_2511533002.equals("*")) { ... * ... } : untuk perkalian.
- else { ... / ... }: Masukkan hasil bagi ke tumpukan.

2.6.3 Output

```
hasil postfix = 20
```

BAB III
KESIMPULAN

Melalui praktikum ini, kita dapat memahami dan mempraktikkan cara pembuatan struktur data Stack (tumpukan) secara manual menggunakan tipe data statis Array pada bahasa pemrograman Java. Secara keseluruhan, tujuan praktikum ini telah tercapai, yaitu memperkenalkan konsep dasar pengorganisasian data berprinsip LIFO (Last In, First Out) melalui perancangan operasi intinya seperti push, pop, dan peek. Hal ini memungkinkan mahasiswa untuk beralih dari sekadar menggunakan pustaka bawaan program ke pemahaman arsitektur tingkat dasar (low-level), di mana kursor indeks dikelola secara mandiri untuk memanipulasi data serta mencegah terjadinya error pada memori seperti overflow dan underflow.

Pemahaman mengenai fondasi dan cara kerja Stack ini menjadi landasan penting untuk topik algoritma maupun pemrosesan komputasi tingkat lanjut, salah satunya diterapkan langsung pada keberhasilan mengevaluasi ekspresi matematika berformat Postfix. Dengan demikian, mahasiswa diharapkan mampu mengembangkan logika program yang lebih kompleks dengan rancangan alur manajemen data sementara yang terorganisir, efisien, dan sesuai dengan cara kerja mesin komputer.